Лаба 6. Хранилище и качество данных

Условия и результат · маршрут показа · справка по коду и n8n

Навигация: стр. 1-3 - суть и показ, стр. 4-8 - справка по коду, стр. 9-10 - узлы n8n. Разделы также доступны в закладках PDF.

Зачем. Данные из разных источников имеют разные поля, пропуски и повторы. Мы приводим их к общей схеме, проверяем допустимость, сохраняем историю и готовим таблицы для анализа. Новый сбор здесь не является целью.

> «Я объединил сохранённые HTML-цитаты и API-цитаты. Исходные данные остаются в raw, очищенные - в clean, аналитические - в mart. Из 452 входных строк получилось 420 цитат. Есть контракт Pydantic и SQLite с историей. При плохом качестве построение mart прекращается».

Что требовалось	Что у нас сделано
≥2 источников, HTML + API, clean ≥300	100 Quotes to Scrape HTML + 320 DummyJSON API = 420 цитат
Raw JSONL, только дозапись, происхождение	URL, UTC-время, источник, версия, статус, исходные поля, проверка SHA-256 прежнего префикса
6 шагов очистки, сравнение 3 форматов	Протокол очистки, CSV/JSONL/Parquet, проверка типов и ID с ведущими нулями
Pydantic: ≥2 field, ≥1 model validator, ≥5 ошибок	3 field_validator, 1 model_validator, 7 примеров отклонения
БД с ограничениями, UPSERT, история, 3 прогона	SQLite, executemany, CHECK/NOT NULL/UNIQUE, история 420 → 420 → 422
≥7 метрик, красный блокирует mart	8 метрик с порогами, контролируемый отказ с кодом 2
3 витрины, партиции по дате, ≥2 графиков	Авторы, источники, теги, 2 даты-партиции, 2 графика
n8n ≥30, брак отдельно, сравнение с кодом	39 входных строк → 34 результата + 3 брака + 2 дубля, расписание, таблица ограничений

Учёт всех строк

452 = 420 принятых + 24 отклонённых + 8 валидных дублей. Добавлены 32 помеченные дефектные копии, 7,08% входа. Они нужны для демонстрации очистки и не создают новые цитаты. В браке: 6 неверных дат, 12 обязательных пропусков, 6 неверных тегов.

Порядок очистки

Текст → бизнес-ключ → типы → пропуски → дубли → бизнес-правила. Удаляются невидимые символы и лишние пробелы, даты приводятся к UTC. Обязательный пропуск отклоняется с причиной, необязательный остаётся NULL. На шаге дублей формируются группы, после проверки выбирается самая свежая валидная запись.

DummyJSON - дополнительный архивный API, API лабы 3 был Open Library. Вариант 3 лабы 6 - цитаты и авторы, а не книги. В итоговом наборе 101 автор и 232 связи цитата-тег.

Лаба 6. Показ по кнопкам

Маршрут на 7-10 минут, без установки и сетевого сбора

1. Открыть ноутбук → **«1. Источники и неизменность raw»**, затем **«2. Очистка и пересборка без сети»**. Покажи два вида источников, 452 → 420 и протокол шести шагов. Пересборка читает локальные файлы.
2. **«3. Сравнение CSV, JSONL и Parquet»**. Покажи строковый ID 000100: автоопределение типов может превратить его в 100. Parquet сохранил строковый тип и UTC-дату.
3. **«4. Контракт и ошибки»**. Покажи модель и семь битых примеров. У каждой отклонённой строки есть поле, причина и исходная запись.
4. SQLite и представления на общей панели. Покажи 420 активных цитат, 24 rejects и три представления. Затем нажми **«Три прогона загрузки»**, результат описан ниже.
5. **«6. Метрики качества и красный светофор»**. Покажи восемь метрик. На общей панели нажми **«Красный светофор»**: ожидается код 2 и неизменность файлов mart. Это успешный тест отказа.
6. **«7. Представления, партиции и два графика»**. Покажи агрегаты и чтение одной партиции: первая партиция содержит 92 строки. Всего в двух партициях 420 строк.
7. Готовый прогон n8n → Normalize fields → Valid record → Remove duplicates → CSV. Покажи 34 результата, отдельный брак и ограничения со следующей страницы.

Что доказывают три загрузки

Прогон	Что произошло	Строк / история
A, первая загрузка	Вставлены 420 записей	420 / 420
B, те же данные	0 обновлений, updated_at не меняется	420 / 420
C, изменены две записи	Обновлены только две, добавлены две версии	420 / 422

Опыт идёт в отдельной демонстрационной БД. Основная warehouse.db сохраняет исходный набор. Нажатия кнопок «Три прогона загрузки» и «Красный светофор» выполняют реальные локальные проверки.

Если попросят код

src/clean.py - очистка, contract.py - правила записи, quality.py - пороги и gate. db/load.py - UPSERT и история, schema.sql - ограничения БД.

Оба опыта выполняются без сети через общую панель. Для просмотра сохранённых результатов в ноутбуке оставь RUN_REBUILD=False. Команды полного повтора есть в README лабораторной.

Лаба 6. Что объяснять

Термины, ограничения no-code и смысл проверок

Что такое raw, clean и mart?

Raw - исходные записи с происхождением. Clean - общая проверенная схема. Mart - готовые аналитические таблицы: по авторам, источникам и тегам. Ошибки можно исправить в обработке и пересобрать clean из raw без обращения к сайтам.

Что такое контракт данных?

Формальные правила допустимой записи. Pydantic проверяет типы, длины, диапазоны и согласованность полей. field_validator проверяет поле, model_validator - связь полей, например ключа и ID источника. Невалидная запись попадает в rejects с причиной.

Как выбран бизнес-ключ?

SHA-256 от строки source + '|' + source_record_id. Источник входит в ключ, чтобы одинаковые ID двух сервисов не столкнулись. Изменение текста сохраняет ключ и позволяет записать новую версию. Хэш имени автора не означает обезличивание.

Что такое UPSERT и идемпотентность?

UPSERT вставляет новый ключ или обновляет существующий. Идемпотентный повтор с теми же данными оставляет тот же результат: нет новых строк, updated_at и история не меняются. История хранит снимок при вставке и при реальном изменении.

Зачем ограничения в БД?

PRIMARY KEY запрещает повтор ключа, NOT NULL - отсутствующее обязательное значение, CHECK - нарушение условия. Они защищают данные и при записи в обход Python. В демонстрации неверная длина текста и дубль ключа отвергаются.

Почему валидность 94,69%, а не 420/452?

Контракт прошли 428 из 452 строк до удаления дублей. Восемь валидных дублей не попали в clean. В журнале 14 альтернатив, шесть из них невалидны и уже входят в 24 rejects. Второй раз прибавлять их к потерям нельзя.

Что получилось в n8n, что осталось в Python

Этап	В n8n	Ограничение
Чтение и очистка	39 строк, нормализация 4 полей	Нет контроля неизменности raw
Проверка и дубли	IF, отдельный rejects.csv, Remove Duplicates	Нет полного Pydantic и почти-дублей
Сохранение	34 строки, CSV заменяется целиком	Нет UPSERT и истории в CSV
Метрики и аналитика	Не реализованы	Светофор и mart выполнены в Python

CSV не поддерживает обновление строки по ключу. n8n умеет UPSERT через узлы БД, но в нашем сценарии выбрана выгрузка CSV. Полный набор Python - 420, n8n показывает подмножество, это допускается ТЗ.

Светофор: проверяет полноту обязательных полей и года рождения, уникальность, валидность raw, долю брака, бизнес-правила, возраст данных и изменение объёма. Год рождения заполнен у 23,81%. У каждой метрики свои пороги. Красный блокирует mart. Возраст считается заново, старый набор со временем может получить другой статус.

Партиция - папка с данными одной даты, чтобы читать нужную часть. JSONL выбран для дозаписи raw, Parquet - для типов и чтения столбцов, CSV - для обмена. График дат показывает даты архивного сбора и учебных копий, а не популярность цитат. В правовом разделе указаны состав данных, ограничения использования и предлагаемый срок хранения до 30 дней после проверки.

Код. Карта файлов и пересборка

Где искать реализацию и в каком порядке она работает

В IDE открой папку **automated_data_collection/lab6_3**. Ctrl+P открывает файл по пути, Ctrl+F находит имя функции. Исходники вынесены в src/ и db/, ноутбук показывает их результаты и выполняет проверки.

Открыть и найти	Что объяснить
src/rebuild.py → rebuild	Управляет всей пересборкой, подробности на этой странице
src/clean.py → clean_rows	Шесть шагов очистки, стр. 5
src/contract.py → Quote	Допустимая запись и валидаторы, стр. 6
db/schema.sql, db/load.py → load	Ограничения, UPSERT, история, стр. 7
src/quality.py → quality, require_mart	Метрики и остановка, стр. 8
nocode/workflow.json	Схема n8n, узлы и показ, стр. 9-10

rebuild(): главный порядок действий

1. **verify_raw()** сравнивает хэши прежних байтов с manifest.json и raw_prefix_manifest.json. Разрешена дозапись, изменение старого префикса вызывает ошибку. Проверяется, что JSONL заканчивается переводом строки.
2. Читаются локальные raw/*.jsonl. **clean_rows(raw, ...)** возвращает чистые записи, отбраковку и протокол. Индексы исходных строк позволяют проверить общий баланс.
3. **quality(...)** рассчитывает метрики, **require_mart(...)** проверяет красные статусы. В этой реализации проверка стоит до сохранения производных clean, БД и mart. Затем отдельно проверяется минимум 300 чистых записей.
4. Сохраняются dataset.parquet, authors.parquet, quote_tags.parquet, rejects.jsonl, дубли и протокол. Каждая исходная строка получает ровно один итог в raw_disposition.json.
5. **benchmark(frame)** сравнивает форматы, **contract_examples(clean)** проверяет семь заведомо битых записей. **load(...)** загружает SQLite, **build_mart(frame)** строит агрегаты, партиции и графики.
6. Повторный **verify_raw()** доказывает, что исходники не изменились во время обработки. Итог сохраняется в clean/rebuild_summary.json.

> «rebuild не скачивает данные. Он координирует обработку локального raw, проверку качества, запись в базу и построение аналитики».

Открыть код: src/rebuild.py. Для обычного показа RUN_REBUILD=False, сохранённые таблицы и проверки доступны в ноутбуке без полной пересборки.

Код. Шесть шагов clean_rows

src/clean.py · как исходная запись становится чистой

clean_rows создаёт рабочие копии записей с _raw_index. Исходные файлы не редактируются. Вложенная **reject** сохраняет индекс, этап, поле, причину и payload, **log** добавляет строку протокола.

Шаг и поиск в файле	Что делает код
1. text(value)	NFKC приводит совместимые символы к общей форме, удаляются zero-width, NBSP становится пробелом, split/join схлопывает пробелы. Теги приводятся к нижнему регистру.
2. normalized_url, quote_key	Удаляются utm_*, fbclid, gclid и фрагмент URL. Ключ цитаты зависит от источника и ID, ключ автора - от имени.
3. datetime.fromisoformat	Дата обязана иметь часовой пояс, затем переводится в UTC. Год рождения - int или None, ID остаётся строкой, text_length считается из текста.
4. missing	Нет текста, автора, URL или ID - запись отклоняется. Неизвестный год рождения остаётся None, среднее не подставляется.
5. by_key, by_near, groups	Кандидаты сортируются по времени. В группы объединяются совпадающие ключи и подписи текста/автора без регистра, пробелов и пунктуации.
6. Quote.model_validate	Проверяются все кандидаты группы. Побеждает самая свежая валидная запись, ошибки остальных сохраняются.

Почему нельзя сразу удалить все старые дубли

Самая свежая копия может быть испорченной. Поэтому шаг 5 формирует группы, шаг 6 проверяет кандидатов от новых к старым. В шести группах свежая копия отклонена и выбрана корректная старая. Счётчик fallback показывает эти случаи.

```
# Внутри шага 6, упрощённый фрагмент
valid.append((row,
    Quote.model_validate(candidate(row)).model_dump(mode='json')))
# После проверки кандидатов:
winner, result = valid[0]
```

candidate(row) переименовывает *_raw в поля общей схемы: text_raw → text, author_raw → author, _source → source. Повторы внутри списка тегов убираются с сохранением порядка. find/union объединяют связанные совпадения в одну группу.

Как читать протокол и баланс

По этапам: **452 → 452 → 452 → 446 → 434 → 420 → 420**. 14 альтернатив в журнале дублей включают 8 валидных дублей и 6 невалидных копий. Последние уже учтены в rejects. Итог без двойного счёта: **452 = 420 + 24 + 8**.

Открыть: clean.py · протокол · распределение исходных строк. Число сообщений rejects может отличаться от числа записей, у одной записи бывает несколько ошибок.

Код. Контракт Quote и ключи

src/contract.py · какие записи допустимы

Quote(BaseModel) - модель Pydantic. Аннотация задаёт тип поля, Field добавляет ограничения. Очистка исправляет представление данных, контракт проверяет результат. Не прошедшая его запись сохраняется в rejects, а не исчезает без объяснения.

```
class Quote(BaseModel):
    text: str = Field(min_length=8, max_length=2000)
    author: str = Field(min_length=2, max_length=150)
    tags: list[str] = Field(max_length=20)
    text_length: int = Field(gt=7, le=2000)
    # Остальные поля см. в src/contract.py
```

Функция / ограничение	Что проверяется
Типы и Field	Ключ цитаты - 64 hex-символа, автора - 24. Источник только quotes_html или dummyjson_api. Год рождения допускает None либо 0-2100.
text_is_content	Нет пробелов по краям, фрагмента <html и символа zero-width. Этот валидатор проверяет очищенный текст цитаты.
tags_are_slugs	Каждый тег состоит из строчных латинских букв, цифр и одиночных дефисов между частями. Пустой список допустим.
utc_date	Дата имеет UTC-смещение 0, будущее дальше чем на 300 секунд отклоняется. Пять минут - допуск к рассинхронизации часов.
consistent, model_validator	Ключ цитаты соответствует source/ID, ключ автора - имени, text_length - длине текста, домен URL - источнику. Схема URL http или https.

Два разных хэша

quote_key(source, source_record_id) считает SHA-256 строки source + вертикальная черта + ID. Так ID двух источников не конфликтуют, изменение текста не меняет ключ.
author_key(name) считает хэш имени после casefold и берёт первые 24 символа. Он связывает таблицы, но не обеспечивает обезличивание.

Что показывать преподавателю

В ноутбуке «4. Контракт и ошибки» сначала проверяются все 420 чистых записей. Затем показаны семь отказов: пустой текст, короткое имя, неверный тег, дата без зоны, неправильный ключ, длина и домен URL. В rebuild.py функция **contract_examples** создаёт эти примеры из копии корректной записи.

> «field_validator проверяет отдельное поле, model_validator проверяет согласованность нескольких полей одной записи».

Открыть: contract.py · семь сообщений об ошибках. model_config extra=ignore означает, что лишние входные поля игнорируются, это не запрет любых дополнительных полей.

Код. SQLite, UPSERT и история

db/schema.sql и db/load.py · как повтор не создаёт дубли

Таблица	Что хранится
authors	Автор, его ключ и необязательный год рождения
quotes	Цитата, ключ автора, происхождение, даты, теги, payload_hash и is_active
quote_tags	Одна строка на пару business_key + tag, ключ составной
rejects	Исходная запись, поле, причина и этап отклонения
quote_history	Payload при вставке и при изменении, ключ цитаты, время и хэш

PRIMARY KEY и UNIQUE(source, source_record_id) запрещают повторы, **NOT NULL** требует обязательное значение, **CHECK** проверяет длины и диапазоны. PRAGMA foreign_keys=ON включает связи таблиц. Индексы author_key, source и tag нужны для группировок и соединений.

load(rows, rejects, database): порядок действий

1. Открывается одно соединение SQLite и транзакция. Создаётся схема, авторы вставляются или обновляются пакетно через **executemany**.
2. Считываются прежние payload_hash и is_active. Для каждой входной записи строится JSON с сортировкой ключей и SHA-256 от содержимого. Это **хэш содержимого**, он отличается от постоянного business_key.
3. В changed попадают новые, изменённые и возвращённые активные записи. UPSERT выполняется только для них. У неизменных не трогаются updated_at, теги и история.
4. Для изменённых цитат заново записываются связи тегов и добавляется снимок quote_history. Отбраковка сохраняется INSERT OR IGNORE по отдельному ключу причины.

```
ON CONFLICT(business_key) DO UPDATE SET ...
WHERE quotes.payload_hash <> excluded.payload_hash
   OR quotes.is_active = 0
```

load принимает полный чистый снимок. Если старого ключа нет во входе, он получает is_active=0, запись и история остаются. Поэтому нельзя передавать произвольные 30 строк как полный набор из 420. Три представления выбирают только is_active=1.

demonstrate(): что доказывает опыт

Создаётся отдельная временная БД. A вставляет 420 строк, B оставляет 420 без изменений и сохраняет updated_at, C добавляет тег demo-change у двух записей. Обновляются ровно две, история растёт 420 → 420 → 422. Затем неверная длина, дубль ключа и NULL текста отвергаются ограничениями БД.

Открыть: load.py → load / demonstrate · schema.sql. В основной warehouse.db остаётся исходный набор, демонстрация изменяет отдельную БД.

Код. Светофор, форматы и mart

src/quality.py и src/rebuild.py · что блокирует плохие данные

quality(raw, clean, rejects, previous_count) возвращает метрики и общий verdict. Для полноты, уникальности и валидности больше - лучше, для брака, возраста и изменения объёма меньше - лучше. Красный означает выход за жёлтый предел.

Метрика	Зелёный	Жёлтый
Обязательные поля	≥100%	≥99%
Год рождения	≥10%	≥5%
Уникальность ключа	≥100%	≥99%
Валидность raw	≥90%	≥80%
Доля брака	≤10%	≤20%
Бизнес-правила	≥100%	≥99%
Медианный возраст	≤30 дней	≤60 дней
Изменение объёма	≤10%	≤20%

Валидность считается для каждой сырой строки после очистки до общей дедупликации: **428/452 = 94,69%**. Доля брака считается по уникальным raw_index: **24/452 = 5,31%**. Возраст - медиана разницы текущего времени и collected_at. Изменение объёма - abs(новый - предыдущий) / предыдущий. Без предыдущего объёма статус bootstrap.

require_mart(report): защитная проверка

```
failed = [r['metric'] for r in report['metrics']
          if r['status'] == 'red']
if failed:
    raise RuntimeError('Mart не обновлён: красные метрики: '
                       + ', '.join(failed))
```

rebuild вызывает эту проверку перед сохранением производных результатов. **--force-red** меняет статус одной метрики в учебной копии оценки. Ожидается RuntimeError, CLI возвращает код 2. Хэши mart до и после совпадают. В ноутбуке показан сохранённый quality.json, полный rebuild рассчитывает возраст заново.

Что делают benchmark и build_mart

benchmark сохраняет один набор в CSV, JSONL и Parquet, измеряет медиану 7 чтений целиком и 3 столбцов, показывает типы даты и ID. JSONL при выборе столбцов всё равно читается целиком. **build_mart** читает SQL-представления, экспортирует их в Parquet и делит снимки по дате collected_at.

При повторе **existing_data_behavior=delete_matching** заменяет затронутые партиции, поэтому копии строк не накапливаются. Чтение с filters выбирает только одну дату. Создаются графики длины цитат и распределения сохранённых срезов.

Открыть: quality.py → quality / require_mart, rebuild.py → benchmark / build_mart. Итоговый красный определяется любой красной метрикой, жёлтый - любой жёлтой при отсутствии красных.

n8n. Чтение, очистка и проверка

Первая половина workflow · что открывать в каждом узле

Открой готовый прогон для просмотра результатов или схему для параметров. Workflow читает сохранённое подмножество **nocode/input.json**, новый сбор с сайтов в нём не выполняется.

Узел	Что показать и объяснить
Manual start	Ручной вход для Execute workflow. Запускает чтение файла.
Daily schedule	Второй вход в тот же путь. Cron 0 8 * * *, timezone Asia/Yekaterinburg. В локальном n8n активен, экспорт JSON сохранён неактивным.
Read source snapshot	Read File(s) From Disk. Читает JSON в бинарное поле data, на выходе один файл, ещё не 39 отдельных записей.
Parse JSON	Extract From File, операция fromJson. Разбирает файл data, массив помещается в records.
Split records	Split Out по records. Один массив превращается в 39 items, по одному объекту на запись.
Normalize fields	Code обрабатывает $input.all(). Приводит текст, автора и ID к строкам, чистит пробелы, нормализует теги и дату. Формирует valid и reason.
Business key	Crypto считает SHA256 от hash_input = source + вертикальная черта + ID. Результат hex записывается в business_key.
Valid record	IF проверяет $json.valid. True идёт к удалению дублей, False - в Reject CSV.

Что означают выражения Code и IF

$input.all() - все входные элементы узла, **$json** - текущая запись. norm выполняет NFKC, удаление невидимых символов, схлопывание пробелов и trim. Date.parse проверяет, что дата разбирается, toISOString записывает её в UTC. ID остаётся строкой, ведущие нули сохраняются.

Normalize fields собирает errors: длина текста вне 8-2000, имя короче 2, неразбираемая дата или неверный slug тега. **valid = errors.length === 0**, reason содержит причины. Теги перед CSV сериализуются JSON.stringify, чтобы список поместился в одну ячейку.

> «Split records меняет единицу обработки: был файл с массивом, стало 39 отдельных записей. IF разветвляет их по результату проверки, Crypto добавляет стабильный ключ».

В этой ветке 39 записей доходят до проверки. На True выходе 36, на False 3. Три отклонённых строки - ошибки данных. Ошибка чтения файла обрабатывается отдельной веткой, описанной на следующей странице.

n8n. Дубли, файлы и показ результата

Вторая половина workflow · все выходы и ограничения

Узел	Что делает и что видно на выходе
Remove duplicates	Удаляет повторы business_key внутри текущего входа. Из 36 валидных items остаётся 34. Это не память о прошлых запусках и не поиск почти-дублей.
Select output fields	Code убирает hash_input, valid и reason из принятой записи. Сохраняются данные и business_key.
CSV → Save CSV	Convert to File создаёт один файл с заголовками и 34 строками, Read/Write Files пишет nocode_result.csv. Один item означает один файл.
Reject CSV → Save rejects	False выход IF превращается в отдельный rejects.csv с тремя строками и причинами отклонения.
Log source error	Error-выход Read source snapshot передаёт ошибку в Set, формируются message и stage.
Error CSV → Save error log	Ошибка чтения источника сохраняется в error_log.csv. Это техническая ошибка файла, она отличается от брака отдельных записей.

Где находятся новые файлы на этом компьютере

Подготовленный n8n использует абсолютные пути: **DataMining/.defense-runtime/outputs/lab6/**, внутри nocode_result.csv и rejects.csv. В экспортированном workflow пути относительные, nocode/ относится к корню lab6_3. Исходная папка сдачи остаётся отдельной копией.

Как показать за две минуты

1. Открой готовый прогон и последовательно нажми Split records, Valid record, Remove duplicates. Покажи переходы **39 → 36 → 34**, в False ветке 3 строки.
2. Нажми CSV и объясни один файловый item. Открой таблицу результата. В Save rejects посмотри путь, отдельный CSV содержит причины.
3. Для нового прогона открой схему и нажми Execute workflow. Он читает локальный JSON без интернета. Проверь завершение обеих веток, количество записей и файл результата.

Что обязательно сказать про ограничения

Повтор заменяет CSV целиком. Это не UPSERT, для обновления по ключу нужен узел БД. В n8n нет полного Pydantic-контракта, fallback для плохой свежей копии, истории, восьми метрик и mart. Code не проверяет все ограничения домена, длины автора, источника и связи полей, которые есть в Python.

Ветка ошибки чтения сохраняет лог, но отдельного Stop And Error в схеме нет. Поэтому завершение workflow само по себе не доказывает правильность данных, проверяй выход нужной ветки. Брак записей при успешном выполнении ожидаем: три дефектных объекта намеренно подготовлены для демонстрации.

Сравнение с Python: ноутбук «8. n8n и сверка с Python» проверяет по business_key, тексту и автору 34 строки сохранённого результата. Пересборка всего Python-набора по расписанию n8n не настроена. Автоматизируется именно это подмножество.